Practice work #10

Review AI-assisted software for risks

Smart Campus Room Availability Checker — Room 304 (Innovation Lab)

Student	Kalmurzaev Nurdaulet
Course	Software Engineering
Date	10 November 2026
AI tool	Claude Code (Claude Opus 5.5), read-only review + reviewed minimal patches
Repository	local git repo `smart-campus-room-checker` — tag `p09-final-checkpoint`, branch `p10/risk-mitigations`
Release decision	**CONDITIONAL**

All data is fictional. No real personal data, secrets or external services are used. Every test result in this report comes from a command that was actually run. The raw outputs are in `docs/practice10/evidence/`.

Task 01 — Prepare the baseline

Project. Smart Campus Room Availability Checker: Room 304 — Innovation Lab, one fictional room, no login and no database. It was built in Bolt.new in A3/Practice #09 as Vite + React + TypeScript + Tailwind, with a Python reference in `python/`.

Note on provenance. The original Bolt.new workspace (`bolt.new/p/72188517`) was not available as a local export. The final Practice #09 project was therefore **reconstructed locally** from the documented A3/Practice #09 design: the same file roles, AC1–AC6, datasets, presets, `can_book()` and its 10 tests. All findings and results in Practice #10 apply to this repository.

Checkpoint before any agent-assisted change: `git tag p09-final-checkpoint` → commit `c9e80c4` on `main`. All Practice #10 changes happen on the separate branch `p10/risk-mitigations`.

Located files

Item	Location	Status
README	`README.md`	present (scope, AC1–AC6, run commands, file list)
AGENTS.md	`AGENTS.md`	present (27 lines: project, commands, style, data)
Tests	`python/test_booking.py` (10 tests)	present for Python only. **No TS/JS tests**, no `tests/` folder
Package scripts	`package.json`	`dev`, `build`, `lint`, `preview`, `typecheck`. **No** `test`
CI / hooks / `.env.example`	—	missing

Existing checks (executed, raw output in `evidence/t1-*.txt`)

Command	Result
`cd python && python3 test_booking.py`	**PASS**: Ran 10 tests, OK
`npm run typecheck`	**PASS**: exit 0
`npm run lint`	**PASS**: exit 0
`npm run build`	**PASS**: vite 5.4.21, built in 1.42 s
`npm test`	**MISSING COMMAND**: `npm error Missing script: "test"` (exit 1)
`npm audit`	**FAIL**: 9 vulnerabilities (8 moderate, 1 high), exit 1

recorded output · ~/Practice10_Kalmurzaev_Nurdaulet/smart-campus-room-checker — Task 01 — baseline checks (p09-final-checkpoint)

```
$ cd python && python3 test_booking.py
test_ac1_exceeds_1440_rejected (__main__.TestCanBook.test_ac1_exceeds_1440_rejected) ... ok
test_ac1_reversed_times_rejected (__main__.TestCanBook.test_ac1_reversed_times_rejected) ... ok
test_ac1_starts_now_rejected (__main__.TestCanBook.test_ac1_starts_now_rejected) ... ok
test_ac1_valid_day_bound_1440 (__main__.TestCanBook.test_ac1_valid_day_bound_1440) ... ok
test_ac2_121_rejected (__main__.TestCanBook.test_ac2_121_rejected) ... ok
test_ac2_exactly_120_allowed (__main__.TestCanBook.test_ac2_exactly_120_allowed) ... ok
test_ac3_blocked_rejected (__main__.TestCanBook.test_ac3_blocked_rejected) ... ok
test_ac4_overlap_rejected (__main__.TestCanBook.test_ac4_overlap_rejected) ... ok
test_ac4_touching_allowed (__main__.TestCanBook.test_ac4_touching_allowed) ... ok
test_ac5_existing_not_mutated (__main__.TestCanBook.test_ac5_existing_not_mutated) ... ok

----------------------------------------------------------------------
Ran 10 tests in 0.000s

OK
[exit code: 0]

$ npm run typecheck --silent
[exit code: 0]

$ npm run lint --silent
[exit code: 0]

$ npm run build --silent
vite v5.4.21 building for production...
transforming...
✓ 1470 modules transformed.
rendering chunks...
computing gzip size...
dist/index.html                   0.49 kB │ gzip:  0.33 kB
dist/assets/index-Ddzrpgpk.css   10.06 kB │ gzip:  2.67 kB
dist/assets/index-CAd0OB12.js   154.46 kB │ gzip: 49.38 kB
✓ built in 1.42s
[exit code: 0]
```

Baseline checks: Python 10/10, typecheck, lint, build

recorded output · ~/Practice10_Kalmurzaev_Nurdaulet/smart-campus-room-checker — Task 01 — located files, scripts, missing npm test

```
$ ls README.md AGENTS.md CONTRIBUTING.md .env.example .github tests src/__tests__ 2>&1; echo; node -e "console.log(JSO
N.stringify(require('./package.json').scripts,null,2))"
ls: cannot access 'CONTRIBUTING.md': No such file or directory
ls: cannot access '.env.example': No such file or directory
ls: cannot access '.github': No such file or directory
ls: cannot access 'tests': No such file or directory
ls: cannot access 'src/__tests__': No such file or directory
AGENTS.md
README.md

{
  "dev": "vite",
  "build": "vite build",
  "lint": "eslint .",
  "preview": "vite preview",
  "typecheck": "tsc --noEmit -p tsconfig.app.json"
}
[exit code: 0]

$ npm test
npm error Missing script: "test"
npm error
npm error To see a list of scripts, run:
npm error   npm run
npm error A complete log of this run can be found in: ~/.npm/_logs/2026-11-10T18_14_05_878Z-debug-0.log
[exit code: 1]
```

Located files and scripts; npm test missing

recorded output · ~/Practice10_Kalmurzaev_Nurdaulet/smart-campus-room-checker — Checkpoint, isolated branch and commits

```
$ git log --oneline --decorate --graph --all
* 16f82e9 (HEAD -> p10/risk-mitigations) docs(practice10): refresh git log evidence
* 78a3dd7 docs(practice10): risk review, register, readiness audit, AGENTS.md audit, mitigations, background-agent spe
c, release decision, evidence
* 03276c3 docs(README): document npm test, src/lib and tests layout
* 1c14022 docs(AGENTS.md): add scope, AC1-AC6 source of truth, forbidden actions, review gate
* e2c42e4 M2 (R1, R2): strict whole-minute parsing and now range validation (AC1)
* 5e22f6a M1 (R3): extract canBook into src/lib/availability.ts, add Vitest parity tests
* c9e80c4 (tag: p09-final-checkpoint, main) Practice #09 final: Room 304 availability checker (baseline checkpoint)
```

Checkpoint tag on main and the isolated mitigation branch

Task 02 — Read-only AI risk review

Student: Kalmurzaev Nurdaulet · **Date:** 2026-11-10 · **Reviewed revision:** tag `p09-final-checkpoint` (commit `c9e80c4`) **AI reviewer:** Claude Code (model Claude Opus 5.5), read-only: it was told not to edit files. Afterwards `git status --porcelain` showed only the new `docs/` evidence folder, and no tracked file had changed.

Prompt used (verbatim)

> Act as a cautious software reviewer. Inspect this Smart Campus Room Availability Checker WITHOUT editing files. Identify risks in correctness, security, privacy, reliability, maintainability, and agent operations. For each risk, cite the exact file/function or missing check, explain the impact, suggest a way to verify it, and state uncertainty. Do not invent test results.

AI output (as returned, lightly formatted)

#	Category	Location	Proposed risk and impact	How to verify	Uncertainty
AI-01	Correctness	`src/App.tsx:95-96`, `parseInt(startInput, 10)`	`parseInt` truncates non-integer input (`"660.5"` → 660, `"6e2"` → 6). A non-whole-minute request can be reported as **Available**, which breaks AC1's "whole minutes" rule.	Enter start `660.5`, end `720` and press Check.	High confidence from reading the code. Browser behaviour of `type=number` not yet observed.
AI-02	Correctness	`src/App.tsx:136-139`, `now` input (`min/max` + `parseInt(...) || 0`)	HTML `min/max` only hint at the range and do not enforce it on typed values. `now` outside 0..1439 (e.g. 2000, -30) is accepted, and an empty field silently becomes 0. The verdict and messages are then wrong (e.g. "after now (33:20)").	Type `2000` and `-30` into *now*, clear the field, then run a check.	High confidence.
AI-03	Maintainability / Reliability	`src/App.tsx:46 canBook` vs `python/booking.py can_book`	The rules exist twice. Only the Python copy has tests (10). The TypeScript copy is not exported and has **no automated tests**, and there is no `npm test` script. The two copies can drift without anyone noticing.	`npm test`; search for `*.test.*`; compare the branch order of both functions.	Drift is not observed today. It is a risk, not a defect.
AI-04	Security (supply chain)	`package.json` / `package-lock.json`	`npm audit` reports 9 vulnerabilities (1 high: `vite <=6.4.3`; moderate: `esbuild` dev server, `minimatch` via `typescript-eslint`).	Run `npm audit`. Check whether the advisories affect the production bundle or only the dev server.	The impact depends on how the dev server is exposed. The static build is likely unaffected.
AI-05	Agent operations	`AGENTS.md`	No forbidden actions (deploy, secrets, new dependencies, network services), no review or approval gate, no verification command before "done", AC1–AC6 not listed, and the scope is not stated.	Read `AGENTS.md` against the course constraints.	Confident (documentation review).
AI-06	Reliability / Agent operations	repository root	No CI workflow, no pre-commit hooks, no `.env.example`. Checks run only when a person remembers to run them.	`ls .github`; check `package.json` for hooks.	Confident.
AI-07	Correctness (AC5)	`src/App.tsx:16 DATASETS`	The bookings arrays are plain mutable objects. Immutability depends on convention only, and no test asserts it for the TypeScript code.	Unit test: call `canBook` and deep-compare the dataset.	`canBook` only reads (`for...of`), so I see no mutation today.
AI-08	Security	`src/App.tsx` rendering	Booking labels and the result message might allow script injection (XSS).	Search for `dangerouslySetInnerHTML` or `innerHTML`.	Low. React escapes text by default.
AI-09	Privacy	`src/App.tsx:16` booking labels	Booking labels may contain personal data about students.	Inspect the data.	Low. The labels look fictional.
AI-10	Reliability	`src/App.tsx:272,298 key={b.start}`	React keys use `start`, so two bookings with the same start would produce duplicate keys and rendering glitches.	Add a dataset with equal starts.	Low. The datasets are fixed and have no equal starts.
AI-11	Correctness	`python/booking.py can_book`	`now` is not range-checked here either, so the Python reference has the same gap as AI-02.	`can_book(0, 60, -30, False, [])` returns True.	Medium. The contract *assumes* a valid `now`.

Statement about tests in the AI output: the reviewer did not run any test during this step. All results quoted later come from commands that were actually executed and recorded in `docs/practice10/evidence/`.

Task 03 — Challenging the AI findings

I chose 9 of the 11 proposed risks across **five** categories: correctness, security, privacy, maintainability/reliability and agent operations. I checked the referenced code myself and ran the commands and UI probes listed below. Labels: **Confirmed** means observed evidence reproduces it. **Plausible / unverified** means the code allows it, but there is no failing observation yet. **Unsupported** means the evidence contradicts the claim.

AI #	Category	My evidence (file / command / observation)	Label
AI-01	Correctness	`evidence/t3-probe-baseline.txt` P1: start `660.5`, end `720` → **"Available — Room 304 is free from 11:00 to 12:00"**. `App.tsx:95 parseInt` drops `.5` without any message. AC1 requires whole minutes.	**Confirmed**
AI-02	Correctness	Probe P3: `now=2000` → "Start (11:00) must be strictly after now (**33:20**)", which is an impossible clock time. P4: `now=-30`, request 0–60 → **Available**. P5: clearing the field shows `0`. `App.tsx:139 parseInt(...) || 0`.	**Confirmed**
AI-11	Correctness	`evidence/t3-verification-commands.txt`: `can_book(0, 60, -30, False, []) = True`. The Python reference does not check `now` either.	**Confirmed** (same root cause as AI-02)
AI-03	Maintainability / Reliability	`evidence/t1-npm-test.txt`: `npm error Missing script: "test"`. No `*.test.*` / `*.spec.*` files exist. `canBook` is a private function inside `App.tsx` and cannot be imported by a test. The Python tests cover only `booking.py`.	**Confirmed** (missing safeguard)
AI-04	Security (supply chain)	`evidence/t1-npm-audit.txt`: 9 vulnerabilities (8 moderate, 1 high). The advisories concern the **Vite/esbuild dev server** and the lint tool chain. `dist/` is static and contains none of these packages at runtime.	**Confirmed** (impact limited to development)
AI-05	Agent operations	`AGENTS.md` has 27 lines. It has no scope statement, AC list, forbidden actions, secrets rule, deployment rule, test-before-done rule or approval gate (see Task 06).	**Confirmed**
AI-06	Reliability / Agent operations	`ls -a .github .husky .env.example` → all missing. There is no CI and there are no hooks.	**Confirmed**
AI-07	Correctness (AC5)	`canBook` only iterates with `for...of`, and timeline sorting uses a copy (`[...bookings].sort`). Probe P6: the bookings list is unchanged after 4 checks. There is no structural protection (`Object.freeze`) and no TS test.	**Plausible / unverified** (no mutation observed)
AI-08	Security (XSS)	`grep -rn "dangerouslySetInnerHTML|innerHTML" src` → no matches. All values are rendered as React text nodes, which React escapes.	**Unsupported**
AI-09	Privacy	The only booking labels are "Robotics Club (fictional)" and "Study Group B (fictional)". The app has no input field for names, no storage and no network calls (`grep fetch/localStorage` → none).	**Unsupported**
AI-10	Reliability	`key={b.start}` (`App.tsx:272,298`). Both datasets have unique starts, so this cannot happen with the current data.	**Plausible / unverified** (low)

No unexecuted test is presented as passed. At this stage no TypeScript test existed. The only executed suite was `python3 test_booking.py` (10/10 OK, `evidence/t1-python-tests.txt`).

recorded output · ~/Practice10_Kalmurzaev_Nurdaulet/smart-campus-room-checker — Task 03 — read-only UI probe + verification commands

```
$ node tools/probe.mjs   # read-only UI probe of the baseline build (p09-final-checkpoint)
P1  start=660.5 end=720 now=540 (AC1 whole minutes)
    -> AvailableRoom 304 is free from 11:00 to 12:00.
P2  start=600.9 end=720 (overlaps 600-660 if read as 600.9)
    -> Unavailable(AC4)Overlaps existing booking 10:00–11:00 (Robotics Club (fictional)).
P3  now=2000 (outside 0..1439), request 660-720
    -> Unavailable(AC1)Start (11:00) must be strictly after now (33:20).
P4  now=-30 (outside 0..1439), request 0-60
    -> AvailableRoom 304 is free from 00:00 to 01:00.
P5  now field cleared -> value shown in #now
    -> now input value = "0"
P6  AC5 bookings list unchanged after 4 checks
    -> unchanged: 10:00–11:00 · [600, 660) · Robotics Club (fictional)
P7  AC4 touching 660-720 (control case)
    -> AvailableRoom 304 is free from 11:00 to 12:00.
page errors: none

$ python3 -c "from booking import can_book; ..."   # AI-11 check, run in python/
can_book(0, 60, -30, False, []) = True

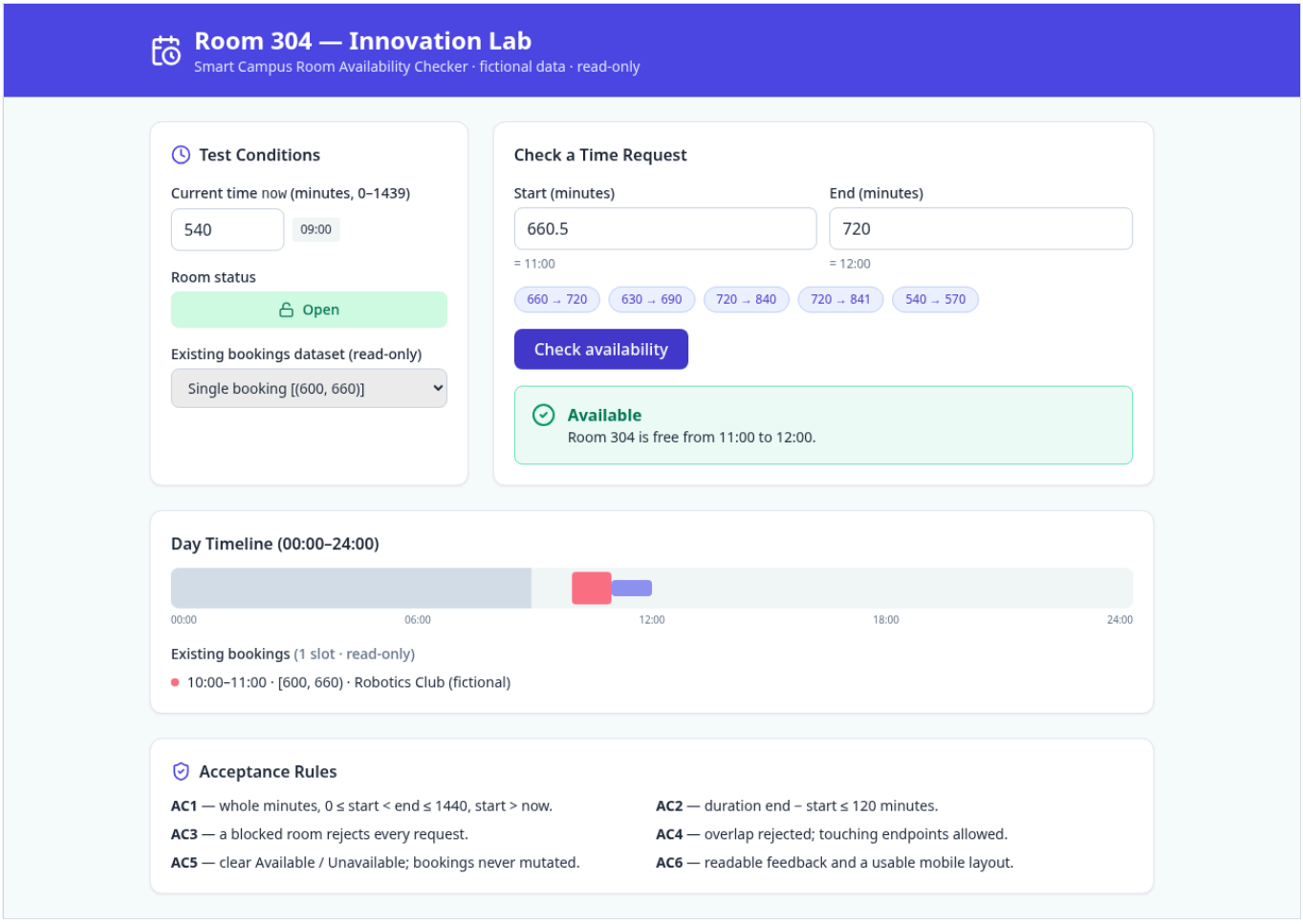
$ grep -rn "dangerouslySetInnerHTML\|innerHTML" src   # AI-08 check
[exit code: 1  -> no matches]

$ grep -rln "" --include=*.test.* --include=*.spec.* . --exclude-dir=node_modules   # AI-03 check
[exit code: 1  -> no TS/JS test files]

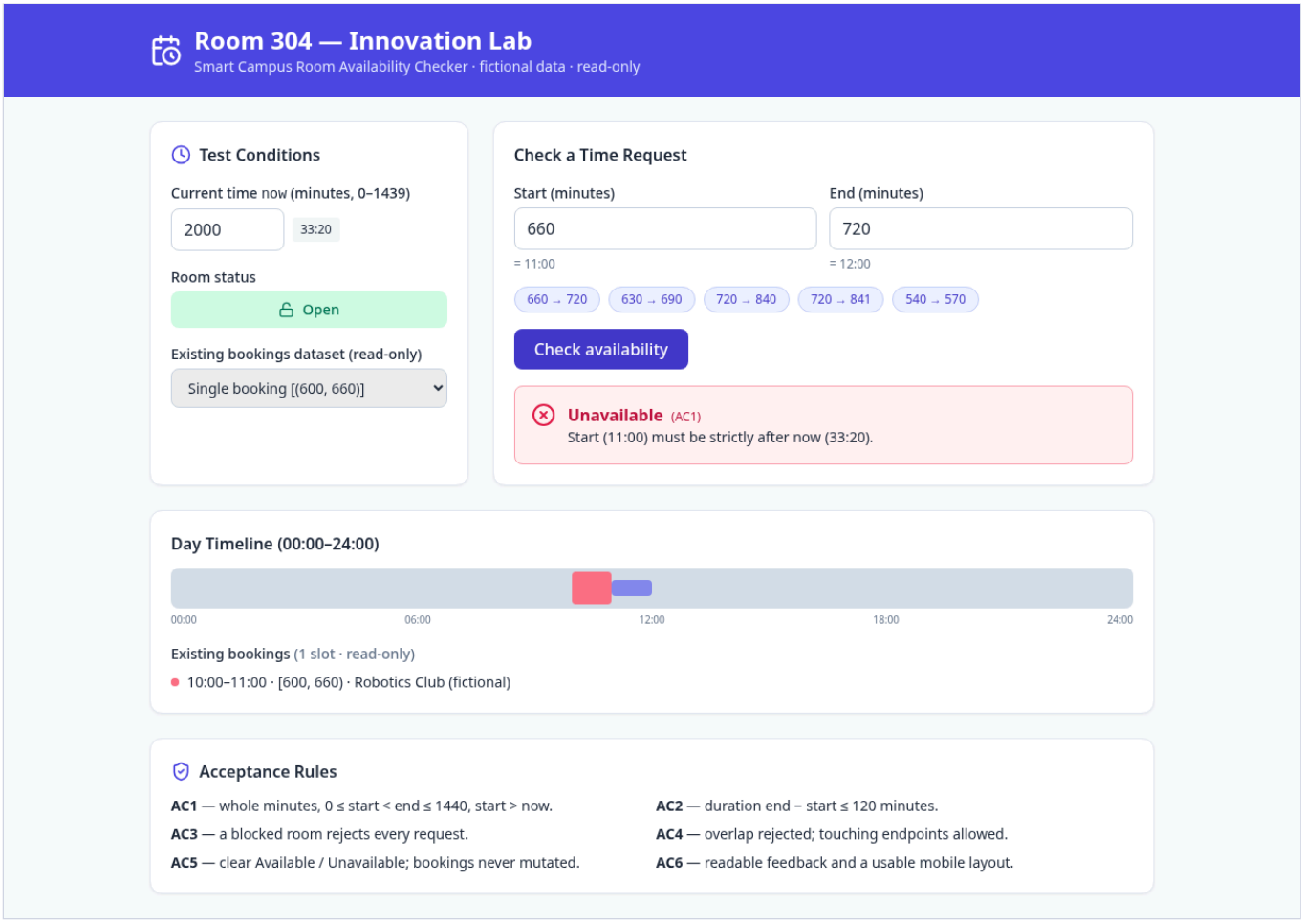
$ ls -a .github .husky .env.example   # AI-06 check
".github": No such file or directory (os error 2)
".husky": No such file or directory (os error 2)
".env.example": No such file or directory (os error 2)

$ git status --porcelain   # after read-only review
?? docs/
```

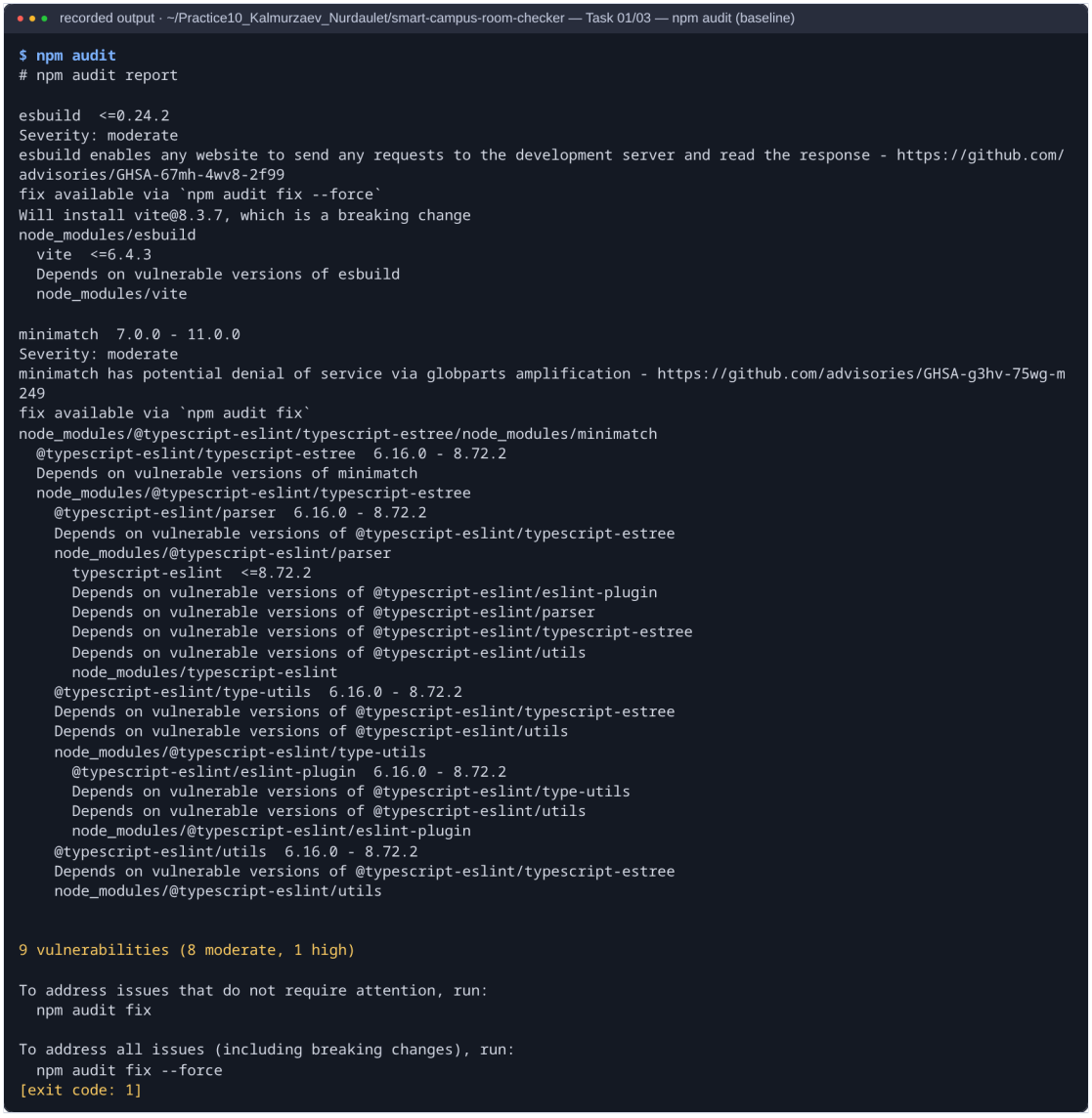
Read-only probe and verification commands (baseline)



Baseline: start 660.5 is shown as 11:00 and reported Available (R1 confirmed)



Baseline: now = 2000 accepted; message shows impossible time 33:20 (R2 confirmed)



npm audit on the baseline (AI-04 / R5)

Task 04 — Risk register

Scoring follows the lecture: **Likelihood** 1 unlikely · 2 possible · 3 likely; **Impact** 1 minor · 2 moderate · 3 serious; **Score = L × I** (1–9). Owner for every engineering action: **Kalmurzaev Nurdaulet** (developer). The release decision is made by the developer and reviewed by the course instructor.

ID	Category	Cause → Event → Consequence	Component	Evidence	Status	L	I	Score	Proposed mitigation	Owner	V
R1	Correctness	`parseInt` truncates input → a non-integer request such as `660.5` is checked as `660` → a wrong **Available** verdict that breaks AC1 (whole minutes).	`src/App.tsx` start/end parsing	Probe P1	Confirmed	2	3	**6**	Strict whole-number parser (`/^\d+$/`). Non-integers are rejected with an AC1 message.	Nurdaulet	F u U
R2	Correctness	The `now` control has no range validation and `|| 0` hides an empty field → `now` outside 0..1439 is used → misleading messages (`33:20`) or a wrong **Available** (`now=-30`).	`src/App.tsx` now input; `python/booking.py`	Probes P3–P5, `can_book(0,60,-30)=True`	Confirmed	2	2	**4**	Validate `now` as a whole minute 0..1439 in `canBook` and `can_book`. Keep the raw field text instead of coercing it to 0.	Nurdaulet	F t F p F
R3	Maintainability / Reliability	The rules are duplicated in TS and Python and the TS copy has no tests → an agent or person edits `App.tsx` and breaks AC1–AC5 → the regression ships unnoticed.	`src/App.tsx` `canBook`; missing `npm test`	`t1-npm-test.txt` (missing script), no test files	Confirmed	3	2	**6**	Extract `canBook` into `src/lib/availability.ts` (no behaviour change). Add Vitest with the 10 A3 cases ported 1:1 (parity) and an `npm test` script.	Nurdaulet	r c p
R4	Agent operations	`AGENTS.md` lacks scope, forbidden actions and review gates → an agent adds a login, a database, secrets or a deployment, or claims "done" without tests → scope and security violations.	`AGENTS.md`	Task 06 audit	Confirmed	3	2	**6**	Minimal AGENTS.md patch: constraints, AC1–AC6, safe commands, forbidden actions, human approval gate.	Nurdaulet	F t a 6 c
R5	Security (supply chain)	Vulnerable `vite@5.4.21`/`esbuild@0.21.5` dev server → a malicious site or local path trick reads dev-server files while `npm run dev` is running → source exposure on the developer machine.	`package.json` dev dependencies	`t1-npm-audit.txt` (1 high, 8 moderate)	Confirmed (dev only)	1	3	**3**	Run the dev server on `127.0.0.1` only. Plan an upgrade to `vite ≥6.4.4` in a separate reviewed change (breaking).	Nurdaulet	r a u
R6	Reliability / Agent operations	No CI and no pre-commit hooks → checks are skipped on some changes → unverified changes are merged.	repository root	`ls .github .husky` → missing	Confirmed	2	2	**4**	Add a CI workflow (`npm ci && npm run lint && npm run typecheck && npm test && npm run build` + Python tests).	Nurdaulet	A r r
R7	Correctness (AC5)	Datasets are mutable objects → a future change mutates bookings during a check → AC5 violation.	`src/App.tsx` `DATASETS`	Code reading; probe P6 unchanged	Plausible	1	2	2	Unit test asserting that the inputs are deep-equal after `canBook`.	Nurdaulet	U
R8	Reliability	`key={b.start}` → duplicate keys if two bookings share a start → rendering glitch.	`src/App.tsx` timeline/list	Code reading	Plausible	1	1	1	Use `${start}-${end}` as the key when datasets change.	Nurdaulet	F v c

Rejected as **unsupported** and not registered as risks: AI-08 (XSS) and AI-09 (privacy). The evidence is in Task 03.

Top two for action

1. **R1 (score 6)** is a user-visible contract violation: the app answers **Available** for an input AC1 forbids. It is confirmed in the UI. → Mitigation **M2** (it also fixes R2, because the same validation step covers `now`).
2. **R3 (score 6, likelihood 3)** blocks verification of every other fix. Without TypeScript tests, R1/R2 regressions cannot be caught. → Mitigation **M1**, done first so that M2 can be regression-tested.

R4 (score 6) is a documentation risk. It is handled by the approved AGENTS.md patch (Task 06), not by a code change.

Task 05 — Agent-readiness audit

Rubric: the Lecture #10 repository readiness levels. **Level 1 Baseline**: formatter, linter, type checks, unit tests, pinned dependencies. **Level 2 Documented**: `.env.example`, architecture notes, `AGENTS.md`. **Level 3 Agent-ready**: CI, hooks, integration checks, reviewable evidence. I use the levels as an audit rubric, not as an automatic certificate.

Checklist (baseline `p09-final-checkpoint` → after `p10/risk-mitigations`)

Area	Check	Baseline evidence	Before	After
README accuracy	Describes scope, AC1–AC6, run commands	`README.md` is accurate for the app. Its "Files" list said `App.tsx` holds "the whole UI and the availability check", and it gave no test command for TS.	Partial	✔ README lists `npm test`, `src/lib`, `tests/`, `AGENTS.md`
Folder layout	Logic separated from UI	All rules inside `src/App.tsx`. `canBook` is private and cannot be tested.	✘	✔ `src/lib/availability.ts` (pure) + `src/App.tsx` (UI)
Documented setup	Install, run, build	`npm install`, `npm run dev`, `npm run build`; Python test command	✔	✔
Formatter	Prettier or similar	none (`node_modules/.bin/prettier` missing, no config)	✘	✘ **gap**
Linter	`npm run lint`	ESLint 9 + typescript-eslint, exit 0 (`t1-lint.txt`)	✔	✔
Type checks	`npm run typecheck`	`tsc --noEmit`, strict mode, exit 0 (`t1-typecheck.txt`)	✔	✔
Unit tests	Automated tests for the shipped logic	Python only (10). TS: `Missing script: "test"` (`t1-npm-test.txt`)	✘ for TS	✔ 32 Vitest tests + 12 Python (`t8-final-*.txt`)
Pinned dependencies	Reproducible install	`package-lock.json` committed. `package.json` uses `^` ranges. README says `npm install`, not `npm ci`.	Partial	Partial (vitest pinned exactly; README still `npm install`)
Build reproducibility	`npm run build` succeeds from the lockfile	exit 0, same bundle hash for CSS (`t1-build.txt`)	✔	✔
Security checks	`npm audit` / secret scanning	Run manually only: 1 high + 8 moderate (dev tool chain)	Manual	Manual: runtime deps 0; dev deps 13 (see R5/R9)
`.env.example`	Env vars documented	none. The app uses **no** env vars (`grep import.meta.env` → none), so this is N/A.	N/A	N/A (stated in this audit)
Architecture notes	Short design description	Only the README file list	✘	Partial (README file roles; no separate notes)
AGENTS.md: scope	Constraints stated	Not stated (Task 06)	✘	✔
AGENTS.md: safe commands	Verification commands	install/dev/build/lint only	Partial	✔ + typecheck, `npm test`, Python tests
AGENTS.md: forbidden actions	Secrets, deploy, deps, refactoring	none	✘	✔
AGENTS.md: review gates	Branch + tests + human approval	none	✘	✔
CI	Workflow running the checks	`.github` missing	✘	✘ **gap**
Hooks	Pre-commit lint/test	`.husky` missing	✘	✘ **gap**
Integration / UI checks	Automated browser checks	none in repo. UI probes exist only as external evidence scripts (`tools/`).	✘	Partial (probes recorded, not in CI)
Reviewable evidence	Commands + outputs, diffs	none	✘	✔ `docs/practice10/evidence/`

Assigned level

Baseline: Level 1 not fully met ("Level 1 – partial"). Lint, strict type checks, a lockfile and a reproducible build existed. But the shipped TypeScript logic had **no unit tests** and there was **no formatter**. Python tests do not verify the code users run.

After mitigations: Level 1 met for tests/lint/types (formatter still missing), Level 2 partially met. `AGENTS.md` now has scope, safe commands, forbidden actions and a review gate. The README is accurate. `.env.example` is not applicable. Architecture notes are minimal. **Level 3 is not met.** There is no CI, no hooks, and no integration tests inside the repo.

Missing evidence / gaps to close

1. A formatter (Prettier) and a `format:check` script. This is the last missing Level 1 item.
2. A CI workflow running `npm ci`, lint, typecheck, `npm test`, Python tests and build on every pull request.
3. Pre-commit hooks (lint + tests).
4. Automated UI/integration tests inside the repo (the `tools/probe.mjs` scenarios turned into Playwright tests).
5. A dependency upgrade (`vite ≥ 6.4.4`, `vitest ≥ 4.1.12`) to clear the dev-tool advisories (R5, R9).
6. A README switch from `npm install` to `npm ci` for reproducible installs.

Task 06 — Audit of the agent playbook (AGENTS.md)

Prompt used (verbatim)

> Review AGENTS.md and project documentation only. Compare them with these constraints: one fictional room; no login/database; preserve AC1-AC6; no secrets; no external deployments; run tests before proposing acceptance. Report missing or conflicting rules. Draft a minimal patch to AGENTS.md, but wait for my approval before applying it.

Documents reviewed: `AGENTS.md` (27 lines) and `README.md` at tag `p09-final-checkpoint`.

Constraint-by-constraint comparison

Constraint	AGENTS.md (before)	README.md	Finding
One fictional room	Mentions Room 304; "Bookings are fictional sample data"	"one fictional campus room"	**Partial.** It does not forbid adding rooms or real data.
No login / database	—	"No login, no database, no backend, no external APIs"	**Missing** in AGENTS.md. An agent reading only AGENTS.md is not told.
Preserve AC1–AC6	—	AC table present	**Missing.** AGENTS.md says "keep rules in sync with `python/booking.py`" but never names AC1–AC6 or which file is authoritative.
No secrets	—	—	**Missing everywhere.**
No external deployments	—	—	**Missing everywhere.**
Run tests before proposing acceptance	Lists install/dev/build/lint only	Python tests only	**Missing.** There is no test command, no typecheck command and no "done" criterion.
Review/approval gate, isolated branch	—	—	**Missing.**

Conflicting or ambiguous rules. I found no direct contradiction. One rule is ambiguous: "*Keep the availability rules in sync with* `python/booking.py`" could be read as "Python is the source of truth". The patch makes README's AC1–AC6 the single source of truth instead. "Do not add booking creation or deletion" is correct and is kept.

Minimal patch (draft — `docs/practice10/agents-md.patch`)

The patch only **adds** 4 short sections and 2 command lines. It deletes nothing and changes no existing rule. Full diff:

```
--- a/AGENTS.md
+++ b/AGENTS.md
@@ -6,11 +6,19 @@
 Smart Campus Room Availability Checker for Room 304 (Innovation Lab). React + TypeScript + Tailwind,
 built with Vite. Python reference in `python/`.

+## Scope (fixed constraints)
+- One fictional room (Room 304) with fictional bookings only. No real names or personal data.
+- No login, no database, no backend, no external APIs or production services.
+- Preserve the acceptance contract AC1–AC6 in `README.md`. That contract is the source of truth;
+  `src/lib/availability.ts` and `python/booking.py` must both implement it.
+
 ## Commands
 - Install: `npm install`
 - Dev server: `npm run dev`
 - Build: `npm run build`
 - Lint: `npm run lint`
+- Type check: `npm run typecheck`
+- Tests: `npm test` and `cd python && python3 test_booking.py`

 ## Coding guidelines
 - Use functional React components and hooks.
@@ -20,3 +28,15 @@

 ## Data
 - Bookings are fictional sample data. Do not add booking creation or deletion.
+
+## Forbidden actions
+- Never add, read or commit secrets, tokens or `.env` files.
+- Never deploy, publish or push to a remote, and never call external services.
+- Do not add dependencies or change the stack without asking first.
+- Do not refactor code that is unrelated to the requested change.
+
+## Review gate
+- Work on a separate branch, never directly on `main`.
+- Before proposing a change as done, run lint, typecheck, `npm test`, the Python tests and `npm run build`.
+  Report the real output; never describe an unexecuted test as passed.
+- A human reviews the diff and approves it before any merge.
```

Approval log

Step	Date	Who	Result
Draft produced, not applied (`git diff AGENTS.md` empty at that point)	2026-11-10	AI agent (Claude Code)	Waiting for approval
Reviewed against the 6 constraints. Applied only after M1, because the patch refers to `npm test` and `src/lib/availability.ts`, which M1 creates.	2026-11-10	Kalmurzaev Nurdaulet	**Approved**

Step	Date	Who	Result
Applied as a separate commit on branch `p10/risk-mitigations`	2026-11-10	Agent, after approval	See `git log`

recorded output · ~/Practice10_Kalmurzaev_Nurdaulet/smart-campus-room-checker — Task 06 — approved AGENTS.md patch applied

```
$ git show --stat 1c14022
commit 1c14022fdff5aed2c4dadf42cd4395c2804d4c7b
Author: Kalmurzaev Nurdaulet <kalmurzaev.nurdaulet@student.invalid>
Date:   Tue Nov 10 23:22:33 2026 +0500

    docs(AGENTS.md): add scope, AC1-AC6 source of truth, forbidden actions, review gate

    Approved by the student after reviewing docs/practice10/agents-md.patch (Task 06).

    Co-Authored-By: Claude Opus 5.5 <noreply@anthropic.com>

 AGENTS.md | 20 ++++++++++++++++++++
 1 file changed, 20 insertions(+)
```

Approved AGENTS.md patch committed separately

Tasks 07 & 08 — Verify safely, mitigate two risks

All work was done on the isolated branch `p10/risk-mitigations`. `main` still points at the untouched checkpoint `p09-final-checkpoint`. Every command below was actually executed. Its full output is in `evidence/`.

Task 07 — Verification log

Step	Command	Result	Evidence
Baseline Python tests	`cd python && python3 test_booking.py`	10/10 OK	`t1-python-tests.txt`
Baseline lint / typecheck / build	`npm run lint`, `npm run typecheck`, `npm run build`	all exit 0	`t1-lint.txt`, `t1-typecheck.txt`, `t1-build.txt`
Baseline TS tests	`npm test`	**not runnable**: `Missing script: "test"`	`t1-npm-test.txt`
Baseline dependency audit	`npm audit`	9 vulnerabilities (1 high, 8 moderate)	`t1-npm-audit.txt`
Baseline UI probe (read-only)	`node tools/probe.mjs`	P1 `660.5` → Available; P3/P4/P5 invalid `now` accepted	`t3-probe-baseline.txt`
Targeted regression tests **before** the fix	`npx vitest run tests/availability.regression.test.ts`	**10 failed / 7 passed** (17)	`t7-regression-before-fix.txt`
Python regression **before** the fix	`python3 test_booking.py`	**FAILED (failures=1)**: `can_book(0,60,-30)` returned True	`t7-python-regression-before-fix.txt`
Manual UI check, desktop 1280 px + mobile 375 px	`node tools/ui-screenshots.mjs before/after`	Feedback text correct for AC1–AC4. **Horizontal overflow 0 px** at 375 px before and after. No page errors.	`t7-manual-ui-check.txt`, screenshots `ui01–ui09`

Tests not run (stated explicitly):

- No real mobile device or Safari/Firefox. Only headless Chromium with a 375 px viewport.
- No screen-reader or accessibility audit.
- No CI run, because no CI exists.
- No test of the original Bolt.new deployment (`*.bolt.host`). Only the local build was tested.
- No load or performance testing; it is not relevant for a static single-page app.

Task 08 — Two mitigations

The top two risks from the register were R3 and R1. R2 has the same root cause as R1 and is closed by the same change.

M1 — R3: TypeScript rules untested and duplicated (commit `5e22f6a`)

- **Request to the agent:** "Minimal patch: move `canBook` and its helpers out of `App.tsx` into a pure module, with no behaviour change. Add a `test` script and port the 10 Python cases 1:1. Do not touch styling or the layout."
- **Diff reviewed first** (`t8-m1-diff.txt`): `App.tsx` had 61 lines deleted and 3 added: one import and two `parseMinutes(...)` calls that replace `parseInt`, with identical behaviour. A new file `src/lib/availability.ts`. `package.json` gained `"test": "vitest run"` and `vitest` pinned at `2.1.9`.
- **Approved:** yes, the change is within scope. **Rejected:** nothing, because the agent proposed no unrelated changes.
- **Behaviour unchanged:** the UI probe before and after M1 is identical, 7/7 (`t8-m1-behaviour-unchanged.txt`).
- **Tests:** 15/15 Vitest passed (`t8-m1-npm-test.txt`). Lint, typecheck and build exit 0 (`t8-m1-checks.txt`).

M2 — R1 + R2: non-integer minutes and invalid `now` accepted (commit `e2c42e4`)

- **Request to the agent:** "Minimal patch so that AC1 is enforced. Start, end and now accept only whole minutes. `now` must be 0..1439 in both `canBook` and `can_book`. An empty `now` field must not become 0. Keep all other rules and messages."
- **Diff reviewed first** (`t8-m2-diff.txt`, 22 insertions / 5 deletions in 4 source files): `parseMinutes` changed from `parseInt` to `/^\d+$/`. There is a new first guard for `now` and `Number.isInteger` for start/end. `App.tsx` keeps the raw `now` text. `booking.py` got a 2-line `now` range check. Plus the regression tests.
- **Rejected / not accepted:** upgrading Vite/Vitest to clear `npm audit` (a breaking, unrelated change, recorded as R5/R9). Also rejected: "clamping" `now` to 0..1439. It would hide invalid input instead of reporting it.
- **First run after the fix:** 1 test still failed. The test expected the message to contain `0–1439`, but the message said "from 0 to 1439" (`t8-m2-first-run-after-fix.txt`). I aligned the message with the UI label "(minutes, 0–1439)".
- **After:** regression suite **17/17**, full suite **32/32** (`t8-m2-regression-after-fix.txt`, `t8-final-npm-test.txt`), Python **12/12 OK** (`t8-final-python-tests.txt`), lint/typecheck/build exit 0 (`t8-final-lint-typecheck-build.txt`). UI probe after: P1–P4 rejected with AC1 messages. P5 keeps the empty field. P6 bookings unchanged. P7 control case still Available (`t8-probe-after.txt`).

Before / after summary

Check	Before (p09-final-checkpoint)	After (p10/risk-mitigations)
`npm test`	missing script	32 passed / 0 failed
Python tests	10 passed	12 passed
UI: start `660.5`	**Available ✘**	Unavailable (AC1) ✔
UI: `now = -30`, 0–60	**Available ✘**	Unavailable (AC1, "now must be 0–1439") ✔
UI: `now = 2000`	"after now (33:20)" ✘	"now must be a whole minute 0–1439" ✔
UI: 660–720 control	Available	Available (unchanged)
Mobile 375 px overflow	0 px	0 px
`npm audit --omit=dev` (runtime)	0	0
`npm audit` (incl. dev)	9 (1 high)	**13 (2 critical, 1 high)**, because Vitest 2.1.9 added 4 advisories (R9)

Residual risk (revised with evidence)

ID	Before	After	Reason
R1	L2 × I3 = 6	**L1 × I1 = 1**	Strict parser and 9 regression cases. Residual: input that browsers transform before it reaches the parser was not tested in non-Chromium browsers.
R2	L2 × I2 = 4	**L1 × I1 = 1**	Guard in both TS and Python, plus regression tests and probes P3–P5.
R3	L3 × I2 = 6	**L2 × I1 = 2**	32 TS tests, 10 of them parity cases. Residual: parity is kept by hand and no CI enforces it (R6).
R4	L3 × I2 = 6	**L2 × I1 = 2**	AGENTS.md patched and approved. Residual: the rules are advisory only and nothing enforces them.
R5	L1 × I3 = 3	3 (unchanged)	Not mitigated. Needs a breaking Vite upgrade.
R6	L2 × I2 = 4	4 (unchanged)	No CI or hooks yet.
R9 (new)	—	**L1 × I2 = 2**	Introduced by M1: `vitest@2.1.9` → `tinypool` (GHSA-5gmw-xhrv-c9v3, GHSA-85c8-ppgw-ccpr), `@vitest/mocker`, `vite-node`. They apply only while tests run locally on trusted code, with no Vitest UI server and no network input. They are not part of the shipped bundle (`npm audit --omit=dev` → 0). Fix: `vitest ≥ 4.1.12`, which requires `vite ≥ 6`, done together with R5.

recorded output · ~/Practice10_Kalmurzaev_Nurdaulet/smart-campus-room-checker — Task 07 — regression tests BEFORE the fix (expected to fail)

```
$ npx vitest run tests/availability.regression.test.ts
npm notice run smart-campus-room-checker@0.0.0 npx
npm notice run 'vitest' run tests/availability.regression.test.ts

 RUN  v2.1.9 ~/Practice10_Kalmurzaev_Nurdaulet/smart-campus-room-checker

 ❯ tests/availability.regression.test.ts (17 tests | 10 failed) 19ms
   × R1 — start/end must be whole minutes (AC1) > parseMinutes("660.5") is rejected (NaN) 8ms
     → expected 660 to be NaN
   × R1 — start/end must be whole minutes (AC1) > parseMinutes("6e2") is rejected (NaN) 1ms
     → expected 6 to be NaN
   × R1 — start/end must be whole minutes (AC1) > parseMinutes("-30") is rejected (NaN) 1ms
     → expected -30 to be NaN
   × R1 — start/end must be whole minutes (AC1) > parseMinutes("12abc") is rejected (NaN) 1ms
     → expected 12 to be NaN
   × R1 — start/end must be whole minutes (AC1) > UI probe P1: 660.5-720 is Unavailable with an AC1 message 1ms
     → expected true to be false // Object.is equality
   × R1 — start/end must be whole minutes (AC1) > a non-integer number passed directly is rejected 1ms
     → expected true to be false // Object.is equality
   × R2 — now must be a whole minute in 0..1439 (AC1) > UI probe P4: now=-30 does not make 0-60 Available 1ms
     → expected true to be false // Object.is equality
   × R2 — now must be a whole minute in 0..1439 (AC1) > UI probe P3: now=2000 explains the invalid now instead of "33:
20" 1ms
     → expected 'Start (11:00) must be strictly after …' to match /0.1439/
   × R2 — now must be a whole minute in 0..1439 (AC1) > now=NaN is rejected 1ms
     → expected true to be false // Object.is equality
   × R2 — now must be a whole minute in 0..1439 (AC1) > now=540.5 is rejected 1ms
     → expected true to be false // Object.is equality

⎯⎯⎯⎯⎯⎯⎯⎯⎯⎯ Failed Tests 10 ⎯⎯⎯⎯⎯⎯⎯⎯⎯⎯

… (171 more lines — full output in evidence/t7-regression-before-fix.txt)

 Test Files  1 failed (1)
      Tests  10 failed | 7 passed (17)
   Start at  23:21:41
   Duration  397ms (transform 51ms, setup 0ms, collect 43ms, tests 19ms, environment 0ms, prepare 79ms)

[exit code: 1]
$ cd python && python3 test_booking.py
test_ac1_exceeds_1440_rejected (__main__.TestCanBook.test_ac1_exceeds_1440_rejected) ... ok
test_ac1_reversed_times_rejected (__main__.TestCanBook.test_ac1_reversed_times_rejected) ... ok
test_ac1_starts_now_rejected (__main__.TestCanBook.test_ac1_starts_now_rejected) ... ok
… (10 more lines — full output in evidence/t7-python-regression-before-fix.txt)
======================================================================
FAIL: test_r2_now_out_of_range_rejected (__main__.TestCanBook.test_r2_now_out_of_range_rejected)
----------------------------------------------------------------------
Traceback (most recent call last):
  File "~/Practice10_Kalmurzaev_Nurdaulet/smart-campus-room-checker/python/test_booking.py", line 44, in test_r2_now_o
ut_of_range_rejected
    self.assertFalse(can_book(0, 60, -30, False, []))
    ~~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
AssertionError: True is not false

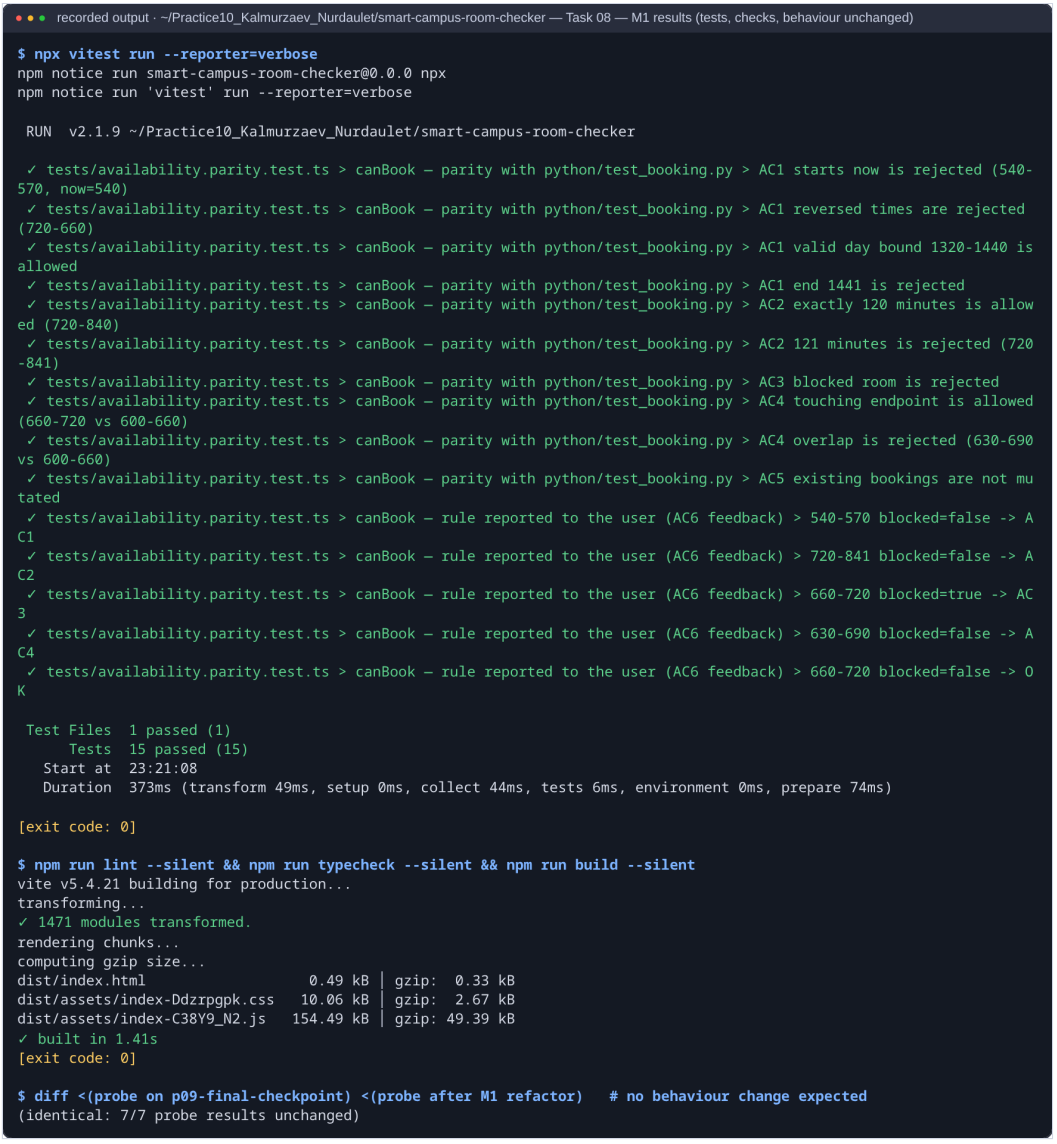
----------------------------------------------------------------------
Ran 12 tests in 0.001s

FAILED (failures=1)
[exit code: 1]
```

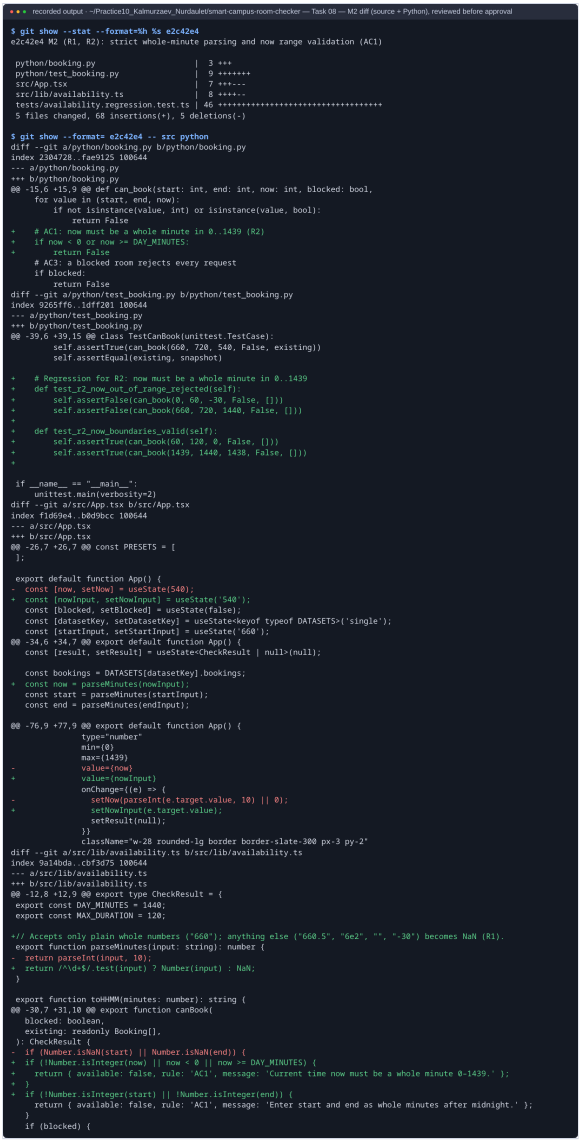
Targeted regression tests fail before the fix (TS 10 failed, Python 1 failed)



M1 diff reviewed before approval



M1: 15/15 tests, checks pass, UI behaviour identical



M2 diff reviewed before approval

recorded output · ~/Practice10_Kalmurzaev_Nurdaulet/smart-campus-room-checker — Task 08 — first run after M2 (1 failure, then message aligned)

```
$ npx vitest run tests/availability.regression.test.ts
npm notice run smart-campus-room-checker@0.0.0 npx
npm notice run 'vitest' run tests/availability.regression.test.ts

 RUN  v2.1.9 ~/Practice10_Kalmurzaev_Nurdaulet/smart-campus-room-checker

 ❯ tests/availability.regression.test.ts (17 tests | 1 failed) 12ms
   × R2 — now must be a whole minute in 0..1439 (AC1) > UI probe P3: now=2000 explains the invalid now instead of "33:20" 6ms
     → expected 'Current time now must be a whole minu…' to match /0.1439/

──────────── Failed Tests 1 ────────────

… (13 more lines — full output in evidence/t8-m2-first-run-after-fix.txt)
       |                     ^
     36|     expect(r.message).not.toMatch(/33:20/);
     37|   });

──────────────────────────────────────[1/1]─

 Test Files  1 failed (1)
      Tests  1 failed | 16 passed (17)
   Start at  23:22:01
   Duration  376ms (transform 45ms, setup 0ms, collect 41ms, tests 12ms, environment 0ms, prepare 78ms)

[exit code: 1]
```

First run after M2: one failing assertion (message wording), fixed



Final: regression 17/17, all 32/32, Python 12/12, lint/typecheck/build OK

recorded output · ~/Practice10_Kalmurzaev_Nurdaulet/smart-campus-room-checker — Task 07/08 — UI probe after fix + manual UI check log

```
$ node tools/probe.mjs   # same read-only UI probe, after M1+M2 (branch p10/risk-mitigations)
P1  start=660.5 end=720 now=540 (AC1 whole minutes)
    -> Unavailable(AC1)Enter start and end as whole minutes after midnight.
P2  start=600.9 end=720 (overlaps 600-660 if read as 600.9)
    -> Unavailable(AC1)Enter start and end as whole minutes after midnight.
P3  now=2000 (outside 0..1439), request 660-720
    -> Unavailable(AC1)Current time now must be a whole minute 0-1439.
P4  now=-30 (outside 0..1439), request 0-60
    -> Unavailable(AC1)Current time now must be a whole minute 0-1439.
P5  now field cleared -> value shown in #now
    -> now input value = ""
P6  AC5 bookings list unchanged after 4 checks
    -> unchanged: 10:00-11:00 · [600, 660) · Robotics Club (fictional)
P7  AC4 touching 660-720 (control case)
    -> AvailableRoom 304 is free from 11:00 to 12:00.
page errors: none

$ PROJECT=<worktree @ p09-final-checkpoint> node tools/ui-screenshots.mjs before screenshots
before 660-720 desktop                      -> AvailableRoom 304 is free from 11:00 to 12:00.
before 660.5-720 desktop                    -> AvailableRoom 304 is free from 11:00 to 12:00.
before now=2000, 660-720                    -> Unavailable(AC1)Start (11:00) must be strictly after now (33:20).
before mobile 375px, 630-690                -> Unavailable(AC4)Overlaps existing booking 10:00-11:00 (Robotics Club
(fictional)).
before mobile horizontal overflow (px)      -> 0
before page errors: none

$ node tools/ui-screenshots.mjs after screenshots   # branch p10/risk-mitigations
after  660.5-720 desktop                    -> Unavailable(AC1)Enter start and end as whole minutes after midnight.
after  now=-30, 0-60                        -> Unavailable(AC1)Current time now must be a whole minute 0-1439.
after  660-720 desktop (control)            -> AvailableRoom 304 is free from 11:00 to 12:00.
after  mobile 375px, 630-690                -> Unavailable(AC4)Overlaps existing booking 10:00-11:00 (Robotics Club
(fictional)).
after  mobile horizontal overflow (px)      -> 0
after  mobile 375px, blocked room, 660-720  -> Unavailable(AC3)Room 304 is blocked — every request is rejected.
after  page errors: none
```

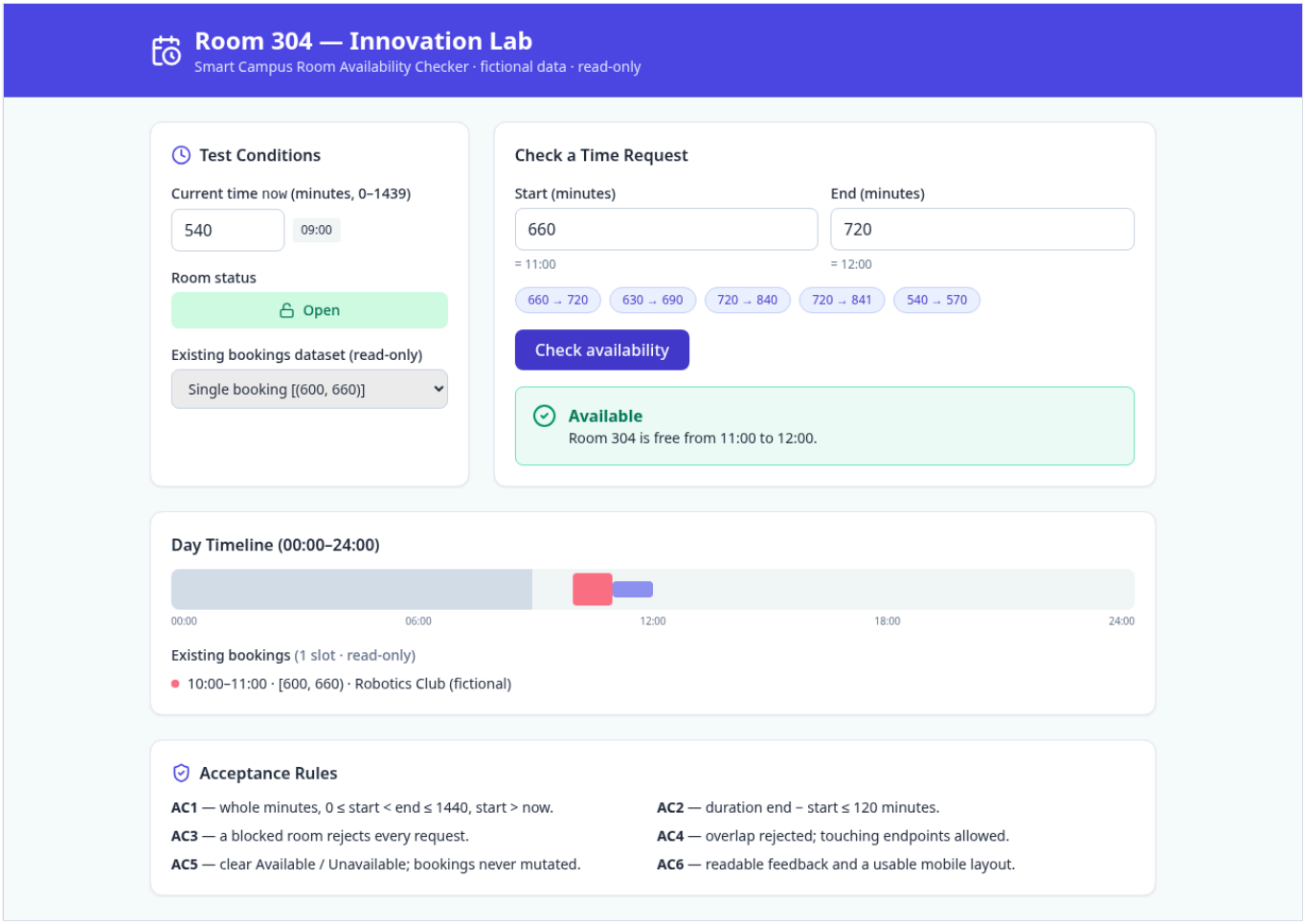
UI probe after mitigations + manual UI check (desktop/mobile, overflow 0 px)

recorded output · ~/Practice10_Kalmurzaev_Nurdaulet/smart-campus-room-checker — Task 08 — npm audit after (runtime 0; dev tool chain R5/R9)

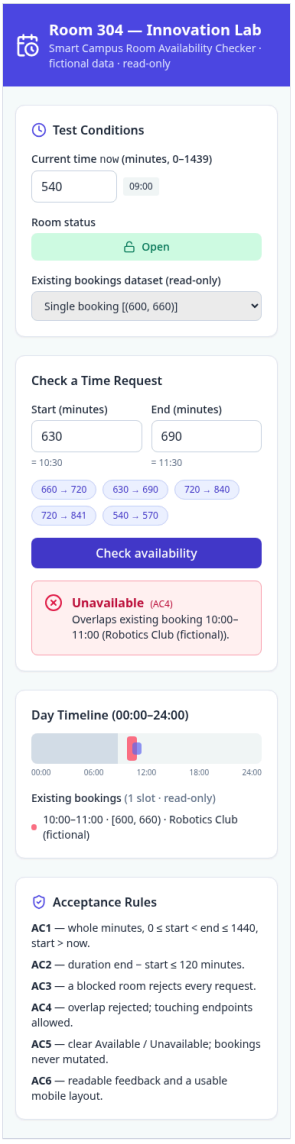
```
$ npm audit --omit=dev   # runtime dependencies that ship to users
found 0 vulnerabilities
[exit code: 0]

$ npm audit 2>&1 | grep -E "vulnerabilities|^[a-z@].*  |Severity"   # all deps incl. dev tooling
@vitest/mocker  <=4.1.11
Severity: moderate
fix available via `npm audit fix --force`
node_modules/@vitest/mocker
esbuild  <=0.24.2
Severity: moderate
esbuild enables any website to send any requests to the development server and read the response - https://github.com/
advisories/GHSA-67mh-4wv8-2f99
fix available via `npm audit fix --force`
node_modules/esbuild
minimatch  7.0.0 - 11.0.0
Severity: moderate
minimatch has potential denial of service via globparts amplification - https://github.com/advisories/GHSA-g3hv-75wg-m
249
fix available via `npm audit fix`
node_modules/@typescript-eslint/typescript-estree/node_modules/minimatch
tinypool  <=2.1.1
Severity: critical
fix available via `npm audit fix --force`
node_modules/tinypool
13 vulnerabilities (10 moderate, 1 high, 2 critical)
```

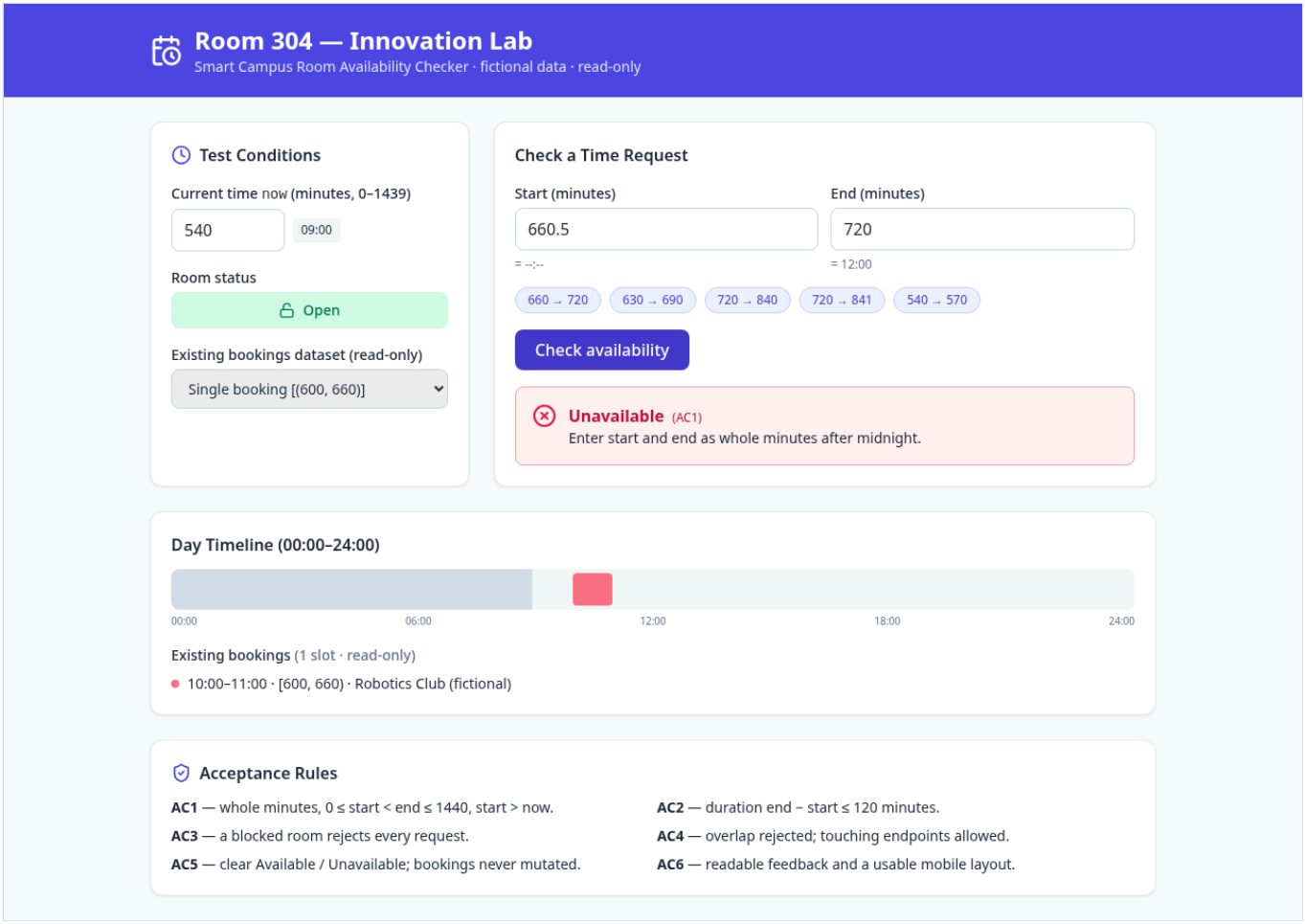
npm audit after: runtime 0; dev tool chain advisories (R5, new R9)



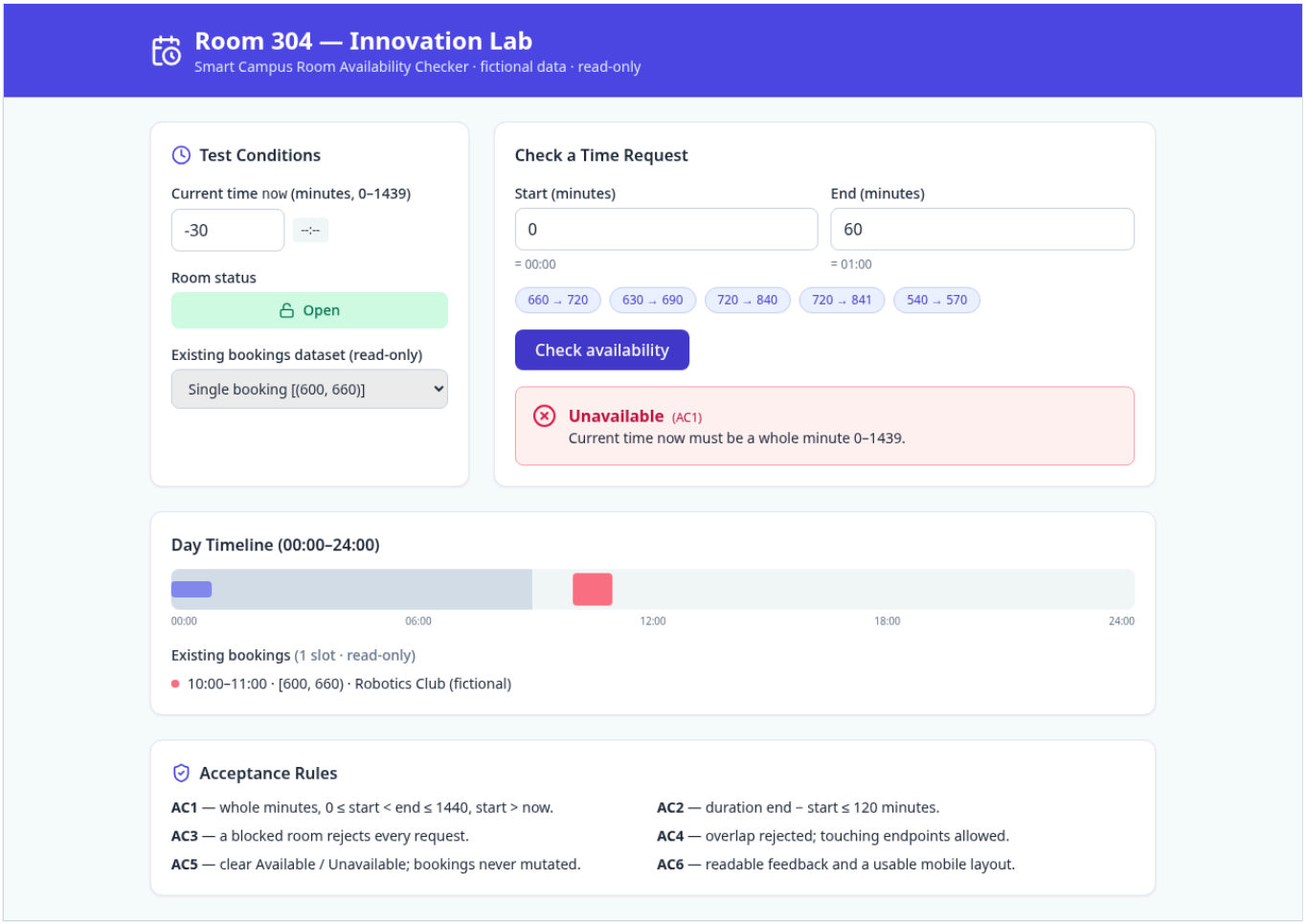
Before — desktop: 660–720 Available (AC4 touching endpoint)



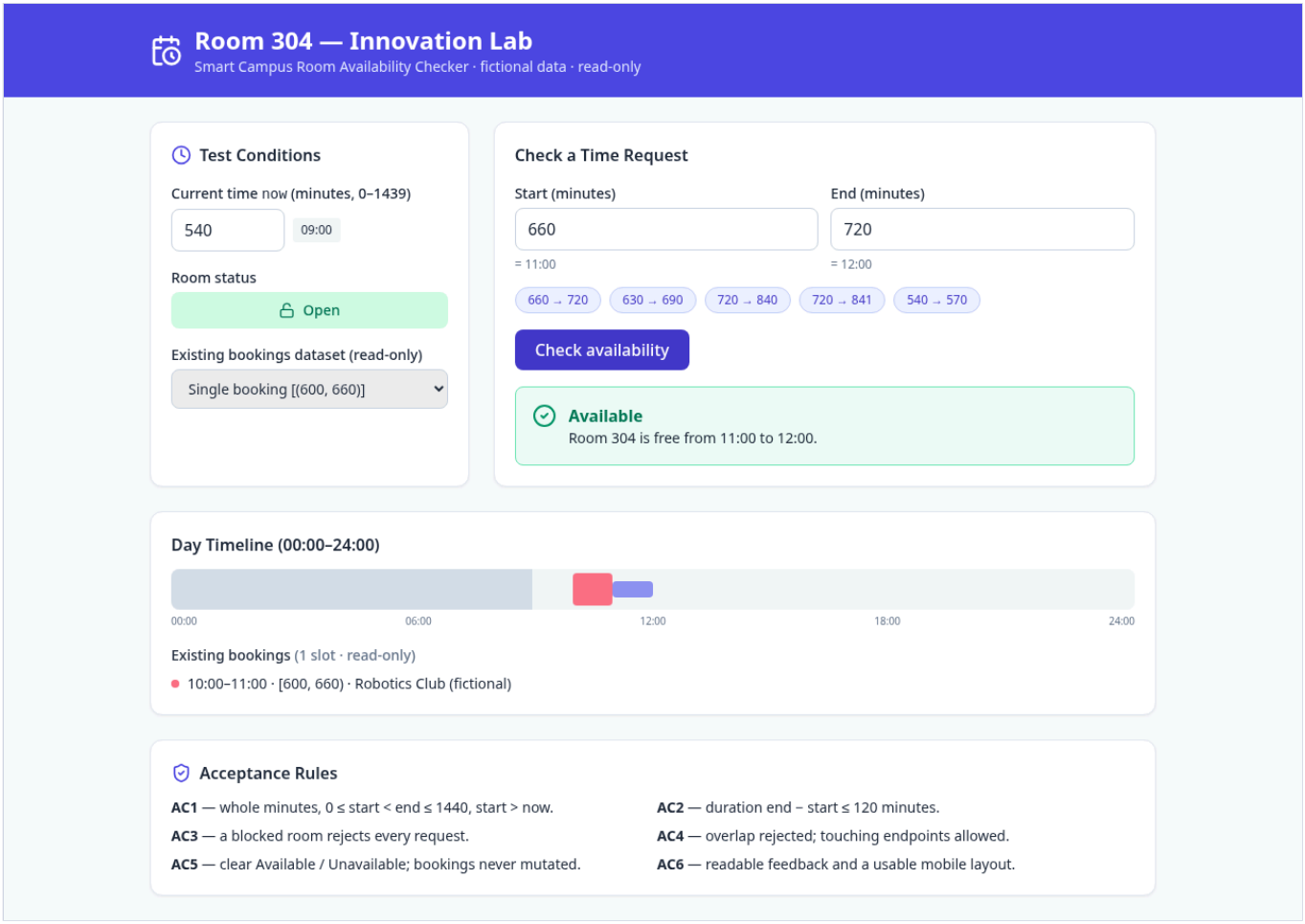
Before — mobile 375 px: overlap rejected (AC4)



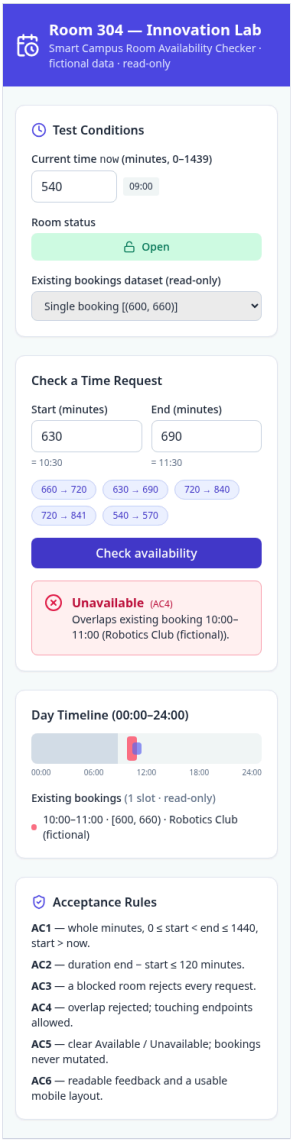
After — 660.5 rejected with AC1 message



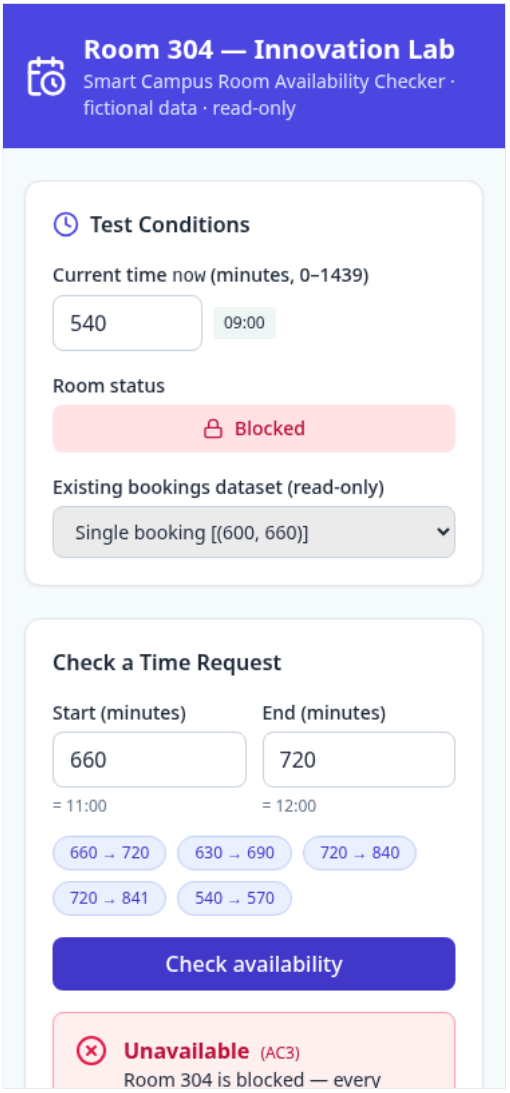
After — now = −30 rejected with AC1 message



After — control case 660–720 still Available



After — mobile 375 px: clear AC4 feedback, no horizontal overflow



After — mobile 375 px: blocked room rejected (AC3)

Task 09 — Background agent thought experiment

Scenario. A coding agent works asynchronously, without a person watching, on the next change for Room 304. It might add Prettier and close readiness gap 1, or clear R5/R9 by upgrading Vite and Vitest.

1. Isolation

- It works on its **own branch** created from `main`, e.g. `agent/upgrade-vite`, inside a **separate git worktree** or a disposable container/sandbox. It never works in my working copy and never on `main`.
- **No credentials** are mounted: no SSH keys, no Git push token, no npm token, no cloud keys, and no `.env` files (the app has none).
- **Network:** only the npm registry, read-only, for `npm ci` and the approved dependency change. Nothing else.
- **Shared resources:** the dev/preview server uses its own port (e.g. 5180 instead of 5173/4173) bound to `127.0.0.1`. There is no shared database because the project has none by design.
- Time and size limits: 30 minutes, at most 10 changed files. If it hits either limit, it stops and reports.

2. Permitted files

Allowed to edit	Read-only	Forbidden
`src/**`, `tests/**`, `python/**`, `package.json`, `package-lock.json` (only for the approved dependency), config files (`vite.config.ts`, `eslint.config.js`, `tsconfig*.json`, new `.prettierrc`)	`README.md` contract table (AC1–AC6), `AGENTS.md`, `docs/practice10/**` evidence	`.git/` internals, `.github/` (CI changes need a separate human-reviewed PR), any `.env*`, anything outside the repository

3. Commands

- **Permitted:** `npm ci`, `npm run lint`, `npm run typecheck`, `npm test`, `npm run build`, `cd python && python3 test_booking.py`, `npx vite preview --host 127.0.0.1 --port 5180`, `git status`, `git diff`, `git add`, `git commit` (on its own branch only).
- **Permitted only with prior approval named in the task:** `npm install <pkg>@<exact version>`.
- **Prohibited:** `git push`, `git merge`, `git rebase main`, `git reset --hard`, `git push --force`, `npm publish`, any deploy command (Bolt/Netlify/Vercel), `curl`/`wget` to external hosts, `rm -rf` outside `dist/` and `node_modules/`, `sudo`, editing the global git config, disabling or deleting tests, adding `.skip` / `.only`.

4. Prohibited actions (scope)

Adding login, a database, a backend, extra rooms, real names or personal data, booking creation or deletion, secrets, analytics or network calls. Changing AC1–AC6. Refactoring outside the task. Claiming a test passed without running it.

5. Required evidence (delivered with the branch)

1. **Diff:** `git diff main...agent/<branch>` plus `--stat`, with every changed file listed and justified in one line.
2. **Test output:** the full raw output of `npm test` and `python3 test_booking.py`, including counts. Failures must be reported, not hidden. Any test not run must be listed with the reason.
3. **Build result:** raw output of `npm run lint`, `npm run typecheck` and `npm run build` with exit codes. For dependency changes, `npm audit` before and after.
4. **Review note:** what was changed and why, which risk-register IDs it affects, new risks introduced (as with R9 in this practice), what is still uncertain, and a log of the tool commands it actually executed.

6. Before merging or deploying

1. A human (me) reads the diff line by line and rejects anything outside scope.
2. A human reruns the checks locally or in CI on a **clean checkout** (`npm ci` → lint → typecheck → test → build → Python tests). The agent's report alone is not proof.
3. A manual UI check of AC6 at desktop width and at 375 px.
4. The risk register is updated: residual scores and any new risks with an owner.
5. The release decision (GO / CONDITIONAL / NO-GO) is recorded and the merge is **explicitly approved by a human**. Only then does a person merge into `main`.
6. **Deployment** is a separate, later, human-only action. The agent never deploys. For this course project nothing is deployed to production services at all.

Release decision: CONDITIONAL

Decision by: Kalmurzaev Nurdaulet · **Date:** 2026-11-10 · **Candidate:** branch `p10/risk-mitigations` (commits `5e22f6a`, `e2c42e4`, `1c14022`, `03276c3` on top of `p09-final-checkpoint`)

Why not GO

The GO criterion is "key checks pass and no unacceptable unresolved risks remain". The key checks do pass. Some risks are still unresolved and need named owners and controls:

- **R5 / R9:** the dev tool chain has 13 advisories: `vite` (1 high), `vitest`/`tinypool` (2 critical), and moderates. They are not in the shipped bundle (`npm audit --omit=dev` → 0 vulnerabilities), but they are real for anyone running `npm run dev` or `npm test`.
- **R6:** there is no CI and there are no hooks. Every check depends on a person running it.
- **Readiness:** Level 1 has no formatter yet, Level 2 is only partly met, and Level 3 is not met.

Why not NO-GO

NO-GO means "serious unmitigated risks or missing critical verification". Neither applies:

- Every **confirmed correctness risk** in the shipped app is mitigated and verified. R1 (score 6 → 1) and R2 (4 → 1) were shown failing in the UI and in 11 regression tests before the fix, and passing after (`t7-*`, `t8-*`).
- The **critical verification exists now:** 32/32 Vitest, 12/12 Python, lint/typecheck/build exit 0, and a manual UI check on desktop and at 375 px with 0 px overflow and no page errors.
- AC5 holds: bookings were unchanged in probe P6 and in the parity test. No secrets, network calls or personal data exist (Task 03: AI-08 and AI-09 are unsupported).

Conditions (owner and control for each known limitation)

#	Limitation	Owner	Control until fixed	Exit criterion
C1	R5/R9 dev-tool advisories	Kalmurzaev Nurdaulet	Run `npm run dev/preview` only on `127.0.0.1`. Never start the Vitest UI server. Run tests only on trusted code.	Separate reviewed PR: `vite ≥ 6.4.4`, `vitest ≥ 4.1.12`, all checks green, and `npm audit` with no high/critical
C2	R6 no CI/hooks	Kalmurzaev Nurdaulet	AGENTS.md review gate: the full check list (lint, typecheck, `npm test`, Python, build) runs by hand before every merge, and the output is attached.	A CI workflow green on a pull request
C3	No formatter (Level 1 gap)	Kalmurzaev Nurdaulet	ESLint and review keep the style consistent	Prettier + `format:check` in CI
C4	Tested only in headless Chromium	Kalmurzaev Nurdaulet	Manual check in one more browser before a demo	Playwright UI tests in the repo (Chromium + Firefox)
C5	Parity between TS and Python is kept by hand	Kalmurzaev Nurdaulet	Parity suite mirrors `test_booking.py` 1:1, and any rule change must touch both	A CI job runs both suites

Remaining uncertainty

- The project was **reconstructed locally** from the Practice #09 / A3 description (see the report). Results apply to this repository. The original Bolt.new workspace was not available for testing.
- The advisory impact assessment (dev-only) is based on `npm audit` metadata and the advisory titles. I did not try to exploit them.
- Browser support beyond Chromium was not verified.

Merge to `main`: allowed only after the human reviewer accepts these conditions. The branch is left unmerged so that the approval stays an explicit human step.

Optional — Agent review checklist prompt

Use this prompt for a **read-only** review of work done by a background coding agent on Smart Campus.

```
You are a READ-ONLY reviewer for a change made by a background coding agent on the Smart Campus Room 304
Availability Checker. Do not edit, create, delete, commit, push, install or deploy anything. Only read files and
run read-only git commands (git log, git diff, git show, git status). If you would need to run tests, list the
exact commands for the human instead of claiming results.

Go through this checklist and answer each item with PASS / FAIL / UNKNOWN, citing evidence
(file:line, commit hash, or a quoted line of recorded output):

1. Isolation: the work is on its own branch/worktree (name it); main is unchanged; no merge, rebase or force-push
   happened (show git log --oneline main..<branch> and main's head).
2. No credentials: the diff adds no secrets, tokens, keys, .env files or credential-like strings; no new
   network calls (fetch, axios, WebSocket) or external URLs.
3. No deployment: no deploy/publish scripts, CI deploy steps, hosting config or npm publish changes.
4. Scope: still one fictional room, no login/database/backend, no booking creation/deletion; AC1–AC6 in README
   unchanged; no unrelated refactoring (list every changed file with a one-line reason; flag any without one).
5. Test and build commands: the agent's report includes the raw output of npm run lint, npm run typecheck,
   npm test, cd python && python3 test_booking.py and npm run build, with exit codes. Mark any missing output
   as UNKNOWN (not PASS). Flag tests that were deleted, skipped (.skip/.only) or weakened.
6. Risk register: docs/practice10/04-risk-register.md (or its successor) was updated for every risk the change
   touches, with revised residual scores and any NEW risk the change introduces (for example dependency advisories).
7. Evidence for each claim: every statement in the agent's summary ("fixed", "tests pass", "no behaviour change")
   is backed by a diff hunk or recorded output. List the claims that have no evidence.
8. Summary of changed files: a table of path | lines +/- | purpose | risk ID.
9. Human approval gate: state clearly that merging requires explicit human approval after the human reruns the
   checks on a clean checkout; end with "Recommendation: APPROVE / REQUEST CHANGES / REJECT" and the reasons.

Rules: do not invent test results; do not claim the agent executed a tool unless its execution log or recorded
output shows it; when evidence is missing, say UNKNOWN and what would settle it.
```